

Database Management System

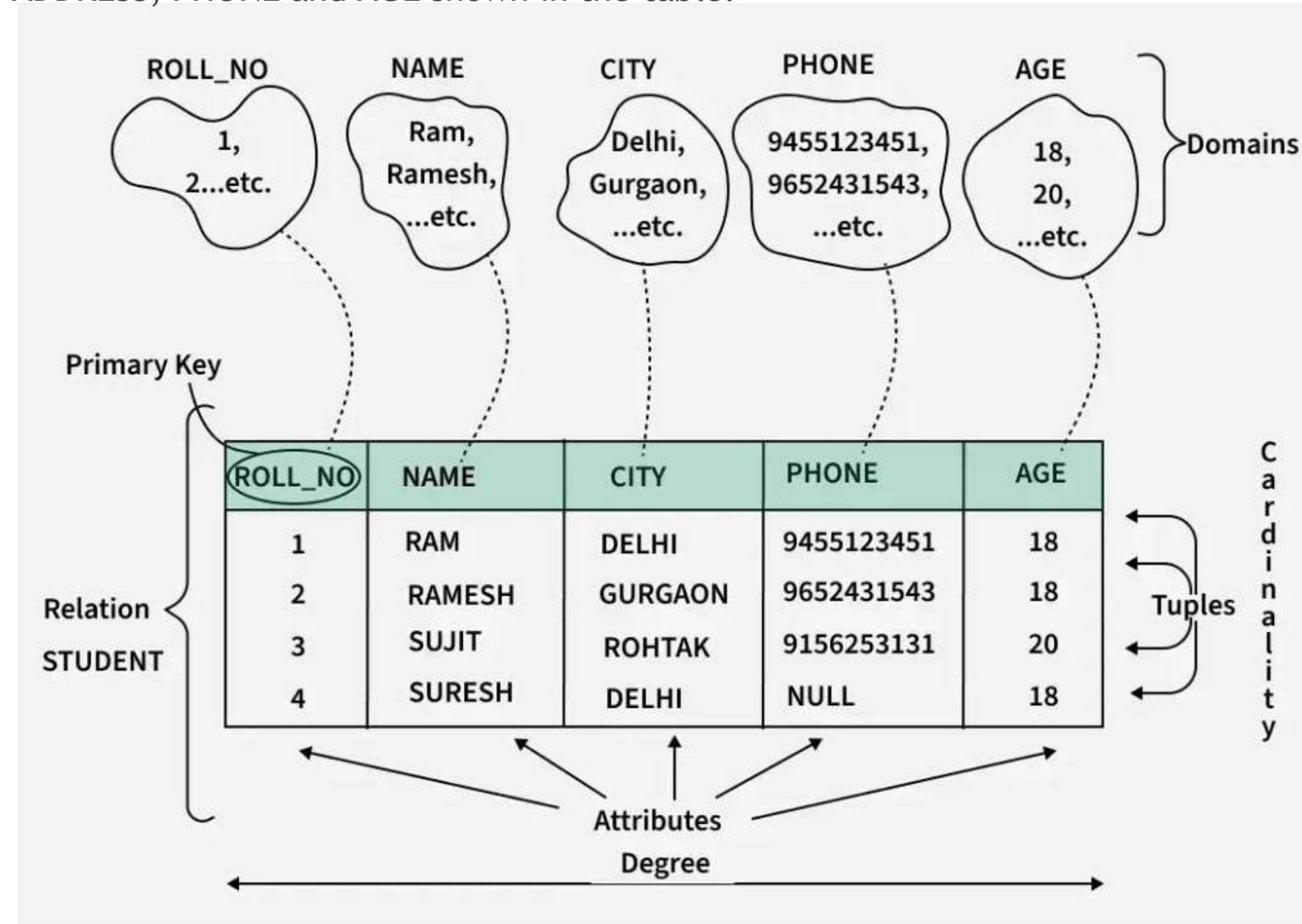
Unit-II

Relational Model in DBMS

The Relational Model organizes data using tables (relations) consisting of rows and columns.

The relational model represents how data is stored and managed in Relational Databases where data is organized into tables, each known as a relation. Each row of a table represents an entity or record and each column represents a particular attribute of that entity. The relational model transforms conceptual designs from ER diagrams into implementable structures. These structures are used in relational database systems like Oracle SQL and MySQL.

Example: Consider a relation STUDENT with attributes ROLL_NO, NAME, ADDRESS, PHONE and AGE shown in the table.



Key Terms in the Relational Model

1.Attribute: Attributes are the properties that define an entity. For example, ROLL_NO, NAME, ADDRESS etc.

2.Relation Schema: A relation schema defines the structure of the relation and represents the name of the relation with its attributes. For example, STUDENT (ROLL_NO, NAME, ADDRESS, PHONE and AGE) is the relation schema for STUDENT. If a schema has more than 1 relation it is called Relational Schema.

3.Tuple: A Tuple represents a row in a relation. Each tuple contains a set of attribute values that describe a particular entity. For example, (1, RAM, DELHI, 9455123451, 18) is a tuple in the STUDENT table.

4. Relation Instance: The set of tuples of a relation at a particular instance of time is called a relation instance. It can change whenever there is an insertion, deletion or update in the database.

5. Degree: The number of attributes in the relation is known as the degree of the relation. For example, The STUDENT relation has a degree of 5, as it has 5 attributes.

6. Cardinality: The number of tuples in a relation is known as cardinality. For example, The STUDENT relation defined above has cardinality 4.

7. NULL Values: The value which is not known or unavailable is called a NULL value. It is represented by NULL. For example, PHONE of STUDENT having ROLL_NO 4 is NULL.

Integrity constraints over relations

▣ Restrictions or sets of rules imposed on the database contents. It validates the quality of the database. It validates the various operations like data insertion, updation, and other processes that have to be performed without affecting the integrity of the data. Mainly Constraints on the relational database are of 4 types

1. Domain constraints
2. Key constraints or Uniqueness Constraints
3. Entity Integrity constraints
4. Referential integrity constraints

1. Domain Constraints

- ▶ Every domain must contain atomic values(smallest indivisible units) which means composite and multi-valued attributes are not allowed.
- ▶ We perform a datatype check here, which means when we assign a data type to a column we limit the values that it can contain. Eg. If we assign the datatype of attribute age as int, we can't give it values other than int datatype.
- ▶ **Example:**

EID	Name	Phone
01	Bikash Dutta	123456789 234456678

2. Key Constraints or Uniqueness Constraints

- ▶ These are called uniqueness constraints since it ensures that every tuple in the relation should be unique.
- ▶ A relation can have multiple keys or candidate keys(minimal superkey), out of which we choose one of the keys as the primary key, we don't have any restriction on choosing the primary key out of candidate keys, but it is suggested to go with the candidate key with less number of attributes.
- ▶ Null values are not allowed in the primary key, hence Not Null constraint is also part of the key constraint.

Example:

EID	Name	Phone
01	Bikash	6000000009
02	Paul	9000090009
01	Tuhin	9234567892

3. Entity Integrity Constraints

- ▶ Entity Integrity constraints say that no primary key can take a NULL value, since using the primary key we identify each tuple uniquely in a relation.
- ▶ **Example:**

EID	Name	Phone
01	Bikash	9000900099
02	Paul	6000000009
NULL	Sony	9234567892

4. Referential Integrity Constraints

- ▶ The Referential integrity constraint is specified between two relations or tables and used to maintain the consistency among the tuples in two relations.
- ▶ This constraint is enforced through a foreign key, when an attribute in the foreign key of relation R1 has the same domain(s) as the primary key of relation R2, then the foreign key of R1 is said to reference or refer to the primary key of relation R2.
- ▶ The values of the foreign key in a tuple of relation R1 can either take the values of the primary key for some tuple in relation R2, or can take NULL values, but can't be empty.
- ▶ **Example:**

EID	Name	DNO
01	Divine	12
02	Dino	22
04	Vivian	14

DNO	Place
12	Jaipur
13	Mumbai
14	Delhi

Enforcing integrity constraints in DBMS

Enforcing integrity constraints in a Database Management System (DBMS) is critical to ensuring that the data stored in the database is accurate, consistent, and reliable. Integrity constraints are rules that data must comply with to be considered valid.

Here are the primary types of integrity constraints and how they are enforced in a DBMS:



1. Domain Constraints

Domain constraints specify that the values of an attribute must be from a specific domain. For example, an age attribute might be constrained to integer values between 0 and 120.

Enforcement:

Data Type Check: The DBMS enforces domain constraints by checking that the data entered matches the specified data type (e.g., integer, string, date).

Range Check: Values are checked to ensure they fall within the specified range or set (e.g., age between 0 and 120).

Pattern Check: For string data, patterns (e.g., regular expressions) can enforce formats like email addresses or phone numbers.

Entity Integrity

Entity integrity is a type of Enforcing integrity constraints, it ensures that each entity (or row) in a database is uniquely identifiable by a primary key. The primary key must be unique and not null.

Enforcement:

Primary Key Constraint: The DBMS enforces uniqueness by disallowing duplicate values in the primary key column(s).

NOT NULL Constraint: Ensures that no null values are allowed in the primary key column(s).

3. Referential Integrity

Referential integrity ensures that a foreign key value always points to an existing, valid record in another table. This maintains the logical consistency of the data.

Enforcement:

Foreign Key Constraint: The DBMS checks that the foreign key value in one table corresponds to a primary key value in the referenced table.

Cascading Actions: The DBMS can enforce referential integrity through cascading actions such as ON DELETE CASCADE or ON UPDATE CASCADE, which automatically propagate changes to foreign keys when the referenced primary key changes.

4. Unique Constraints

Unique constraints ensure that all values in a column or a group of columns are unique across the table.

Enforcement:

Unique Index: The DBMS creates a unique index on the column(s) specified, disallowing duplicate values.

5. Check Constraints

Check constraints enforce a condition that each row in the table must satisfy. These are custom rules defined by the user.

Enforcement:

Check Clause: The DBMS evaluates the condition specified in the CHECK clause whenever data is inserted or updated. If the condition is violated, the operation is rejected.

6. NULL Constraints

NULL constraints specify whether a column can accept NULL values or not.

Enforcement:

NOT NULL Constraint: The DBMS ensures that columns specified with NOT NULL cannot store NULL values.

Querying Relational Data in DBMS

A relational database query is a question about the data, and the answer consists of a new relation containing the result. For example, we might want to find all students AGE less than 18 or all students enrolled in particular course.

The SELECT statement is used to fetch the data from a database table which returns this data in the form of a result table. These result tables are called result-sets.

Syntax in Mysql

```
SELECT column1, column2, ...  
FROM table_name;
```

```
SELECT * FROM table_name;
```

The symbol '*' means that we retain all fields of selected tuples in the result.

We can retrieve rows corresponding to students who are younger than 18 with the following SQL query:

Example:

```
SELECT * FROM Students WHERE age < 18;
```

The condition `age < 18` in the WHERE clause specifies that we want to select only tuples in which the age field has a value less than 18.

In addition to selecting a subset of tuples, a query can extract a subset of the fields of each selected tuple. we can compute the `student_id` and `First_name` of students who are younger than 18 with the following query:

Example:

```
SELECT ID,FirstName FROM Students WHERE age < 18;
```

Mapping from ER Model to Relational Model

ER models are converted into the relational model because RDBMS can easily implement relational models like Oracle, My SQL, etc.

ER diagrams mainly consist of

1. **Entity:** An entity is a real-world object having some attributes.
2. **Relationship:** It is an association among different entities.

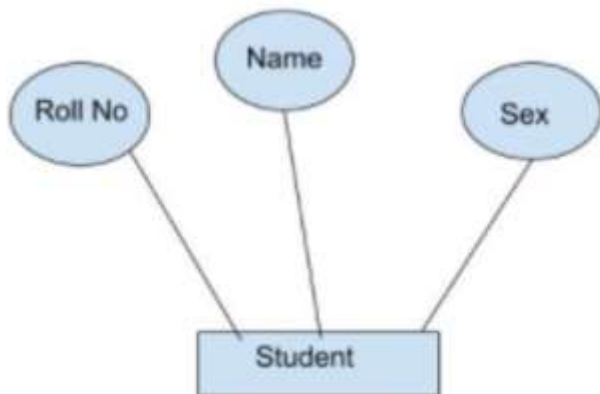
Rules

CASE1: For Strong Entity Set

A strong entity set having only simple attributes or composite attributes will require only one table in the relational model.

The simple attribute of the table will be the attribute of the entity set. Simple attributes of the composite attributes are considered and not composite attributes themselves. The primary key of the resulting table will be the key attribute of the entity set.

Examples



<u>Roll No</u>	Name	<u>Sex</u>

FIG1: Strong entity with Simple Attributes

CASE 2: For Strong Entity Set With Multi-Valued Attributes

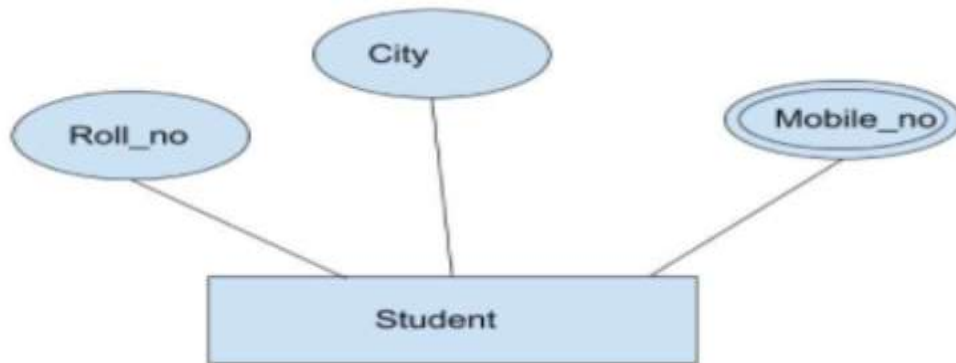
A strong entity set having any number of multi-valued attributes will require two tables in the relational model.

One table will consist of all the simple attributes with the primary key.

Another table will contain the primary key and all the multi-valued attributes.

Example

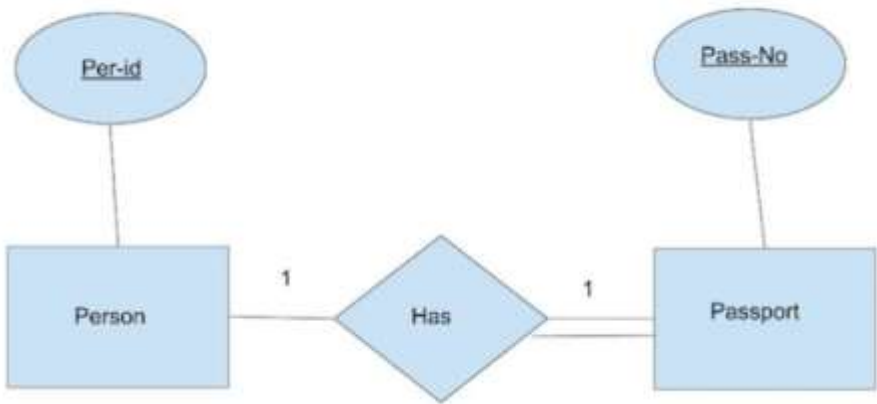
In this example, Mobile_no is a multivalued attribute. Here, Roll_no is the primary key. So we made a table using primary key Roll_no and remaining simple attributes and other tables having attributes Roll_no and Mobile_no.



<u>Roll_no</u>	City

<u>Roll_no</u>	Mobile_no

CASE 3: Binary Relationship with 1:1 cardinality with the total participation of an entity

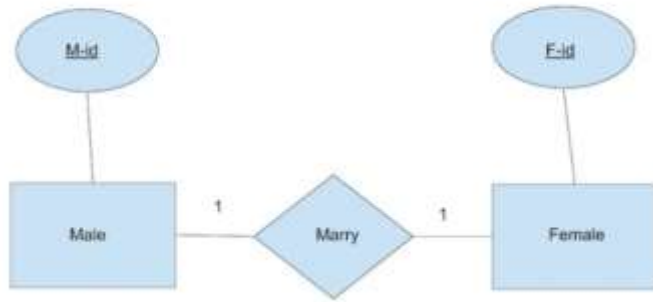


Person		Has		Passport	
<u>Per-Id</u>	Other Person Attribute	<u>Per-Id</u>	<u>Pass-No</u>	<u>Pass-No</u>	Other PassportAttribute
PR1	-	PR1	PS1	PS1	-
PR2	-	PR2	PS2	PS2	-
PR3	-				

In this example, a person can have 0 or 1 passport number. A single person owns each passport shown., So it is one-to-one (1:1) cardinality with the full participation of the Passport entity.

Firstly, we will convert every entity and relationship to tables. We have three tables corresponding to two entities and one for the relation between the entities. Person table and Passport table correspond to Person and Passport entity respectively. The Has table shows the relationship between Passport and Person table. So it will take attribute Per-Id from Person and Pass-No from Passport.

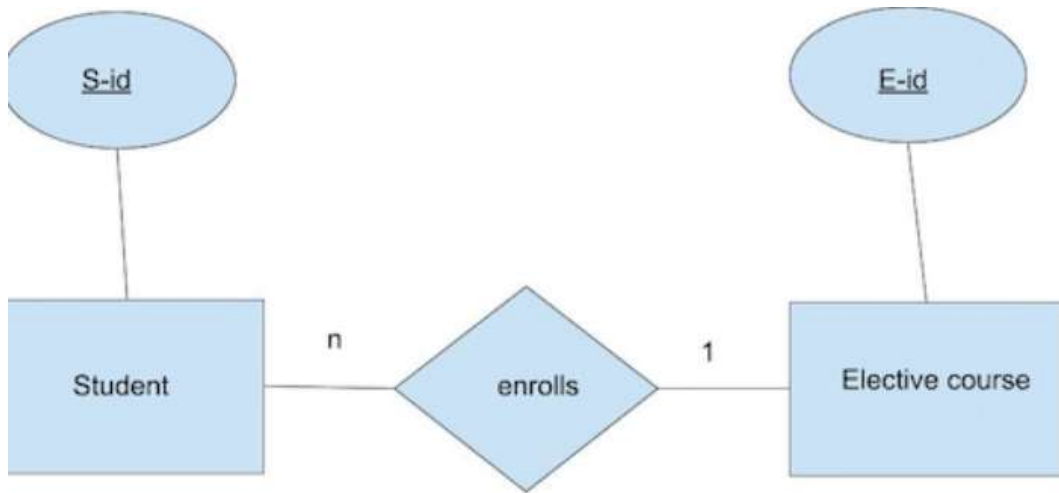
CASE 4: Binary Relationship with 1:1 cardinality and partial participation of both entities



Male		Marry		Female	
<u>M-Id</u>	Other Male Attribute	<u>M-Id</u>	F-Id	<u>F-Id</u>	Other FemaleAttribute
M1	-	M1	F2	F1	-
M2	-	M2	F1	F2	-
M3	-			F3	-

In this example, a male can marry 0 or 1 female and vice-versa. So it is one-to-one cardinality with partial participation from both male and female entities. We will have three tables corresponding to two entities and one for the relation between the entities. The male and Female table correspond to the Male and female entities, respectively. The Marry table shows the relationship between the Male and Female table. So it will take attribute M-Id from Male and F-Id from Female.

CASE 5: Binary Relationship having n: 1 cardinality

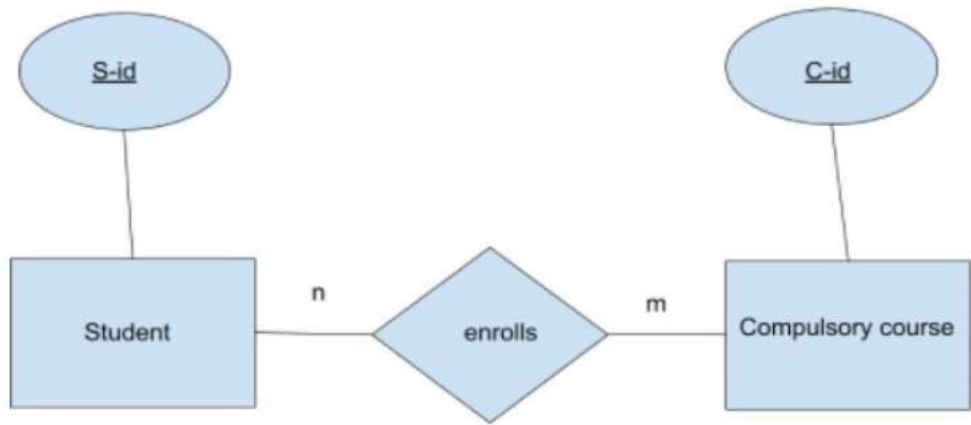


Student		Enrolls		Elective_Course	
<u>S-</u> <u>Id</u>	Other Student Attribute	<u>S-</u> <u>Id</u>	E- Id	<u>E-</u> <u>Id</u>	Other Elective CourseAttribute
S1	-	S1	E1	E1	-
S2	-	S2	E2	E2	-
S3	-	S3	E1	E3	-
S4	-	S4	E1		

As we can see from the given table, E-Id is unique in Enrolls Table. So it can be a key of Enrolls.

In this example, each student can enroll in only one elective course, but multiple students can be enrolled for an elective course. We will have three tables corresponding to two entities and one for the relation between the entities. The Student table and Elective_Course table correspond to the Student and Elective_Course entity. The Enrolls table shows the relationship between the Student and Elective_Course table. So it will take attribute S-Id from Student and E-Id from Elective_Course.

CASE 6: Binary Relationship having m: n cardinality

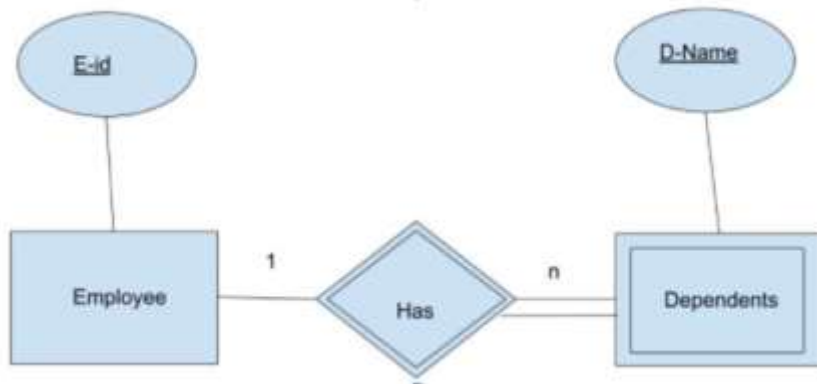


Student		Enrolls		Compulsory_Courses	
<u>S-Id</u>	Other Student Attribute	<u>S-Id</u>	<u>C-Id</u>	<u>C-Id</u>	Other Compulsory CourseAttribute
S1	-	S1	C1	C1	-
S2	-	S1	C2	C2	-
S3	-	S3	C1	C3	-
S4	-	S4	C3	C4	-
		S4	C2		
		S3	C3		

In this example, each student can enroll in more than one compulsory course, and for each compulsory course, one or more students can enroll.

We will have three tables corresponding to two entities and one for the relation between the entities. The Student table and Compulsory_Courses table correspond to the Student and Compulsory Courses entity. The Enrolls table shows the relationship between the Student and Compulsory_Courses table. So it will take attribute S-Id from Person and C-Id from Compulsory_Courses.

CASE 7: Binary Relationship having a weak entity



Employee		Has		Dependents		
<u>E-Id</u>	Other Employee Attribute	<u>E-Id</u>	<u>D-Name</u>	<u>D-Name</u>	<u>E-Id</u>	Other DependentsAttribute
E1	-	E1	RAM	RAM	E1	-
E2	-	E1	SRINI	SRINI	E1	-
E3	-	E2	RAM	RAM	E2	-
		E3	ASHISH	ASHISH	E3	-

In this example, an employee can have many dependents, and one dependent can depend on one employee. A dependent does have no existence without an employee. So dependent will be a weak entity whose participation will always be total. The weak entity does not have its key. So its key will be a combination of its identifying entity (E-Id of Employee in this case) and its partial key (D-Name). We will have three tables corresponding to two entities and one for the relation between the entities. The Employee table and Dependents table correspond to the Employee and Dependent entity. The Has table shows the relationship between the Employee and Dependents table. So it will take attribute E-Id from Employee and D-Name from Dependents.

Introduction to views. Creation, altering and dropping (destroying) of tables and views

A view is a table whose rows are not explicitly stored, a view is a virtual table based on the result-set of an SQL statement. A view can contain all rows of a table or select rows from a table. A view can be created from one or many tables which depends on the written SQL query to create a view.

A view is generated to show the information that the end-user requests the [data](#) according to specified needs rather than complete information of the table.

Advantages of View over [database](#) tables

Using Views, we can join multiple tables into a single virtual table.

Views hide data complexity.

In the database, views take less space than tables for storing data because the database contains only the view definition.

Views indicate the subset of that data, which is contained in the tables of the database.

Creating Views

Database views are created using the CREATE VIEW statement. Views can be created from a single table, multiple tables or another view. To create a view, a user must have the appropriate system privilege according to the specific implementation.

Syntax in Mysql

```
CREATE VIEW view_name AS SELECT column1, column2, ... FROM table_name WHERE condition;
```

Example: `CREATE VIEW Students_CSE AS SELECT Roll_no, Name FROM Students WHERE Branch = 'CSE'`

Updating a View

A view can be updated with the CREATE OR REPLACE VIEW statement.

Syntax in Mysql

```
CREATE OR REPLACE VIEW view_name AS SELECT column1, column2, ... FROM table_name WHERE condition;
```

The following SQL adds the "Mobile" column to the "Students_CSE" view:

Example: `CREATE OR REPLACE VIEW Students_CSE AS SELECT Roll_no, Name, Mobile FROM Students WHERE Branch = 'CSE';`

CREATE VIEW defines a view on a set of tables or views or both.

REPLACE VIEW redefines an existing view or, if the specified view does not exist,

Deleting a row in a view

Deleting rows from a view is also as simple as deleting rows from a table. We can use the DELETE statement of SQL to delete rows from a view.

Syntax in Mysql

DELETE FROM view_name WHERE condition;

Example:DELETE FROM Students_CSE WHERE Name="ram";

Querying a View

We can query the view as follows

*Syntax in Mysql*SELECT * FROM view_name

Example:SELECT * FROM Students_CSE;

Dropping a View

In order to delete a view in a database, we can use the DROP VIEW statement.

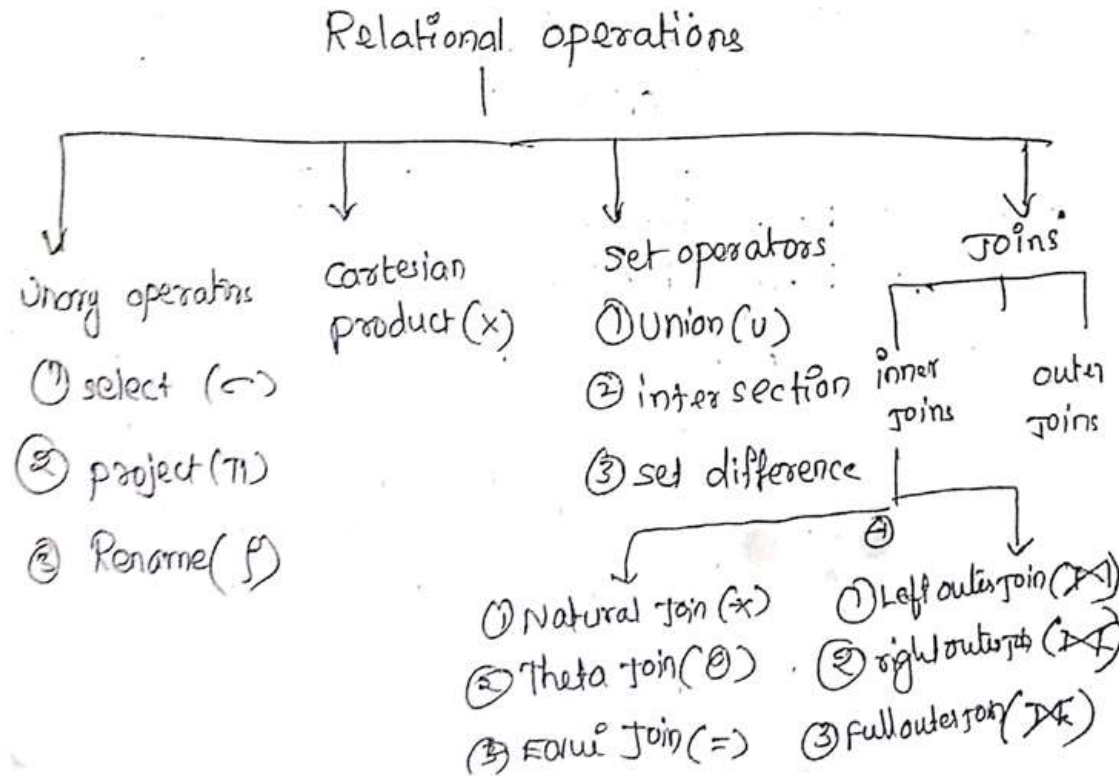
Syntax in Mysql DROP FROM view_name

Example:DROP FROM Students_CSE;

Introduction to Relational Algebra in DBMS

Relational Algebra is a formal language used to query and manipulate relational databases, consisting of a set of operations like selection, projection, union, and join. It provides a mathematical framework for querying databases, ensuring efficient data retrieval and manipulation.

- Relational algebra serves as the mathematical foundation for SQL query.
- SQL queries are based on relational algebra operations, enabling users to retrieve data effectively.



Unary Operator: Operators which are performed on single table is called unary operators

1. Selection(σ)

The [Selection Operator](#) is used to filter rows from a table based on certain given condition. It is similar to where clause in [SQL Query](#).

Example: student

Name	Rollno	Age
sai	21	18
Shiva	22	19
Mohan	23	17
Nikil	21	18

Syntax

Syntax $\rightarrow$ (Relation)
(select condition)
Ex1 $\rightarrow$ (student)
(rollno = 22)

Output:

Name	Rollno	Age
shiva	22	19

2. Projection(π)

A projection operator is used to select specific columns from a table while discarding the other columns it allows you to extract only the information you need from a table, which improves efficiency.

Query: Select name of the student whose age is 19

Syntax

$\pi_{A_1, A_2 \dots A_n} (\text{Relation})$

Ex

student

name	rollno	age
Sai	21	18
shiva	22	19
Akil	23	17
mohan	24	18

$\pi_{(\text{name}, \text{age})} (\text{student})$

output

name	age
Sai	18
shiva	19
Akil	17
mohan	18

$\pi_{\text{name}} (\sigma_{(\text{age}=19)} (\text{student}))$

o/p

name

shiva

3.Rename

Rename (p) :The rename operations is used to assign new names to tables or columns

syntax
To rename table $\rightarrow$ $\int$ (new table name) (old table name)

To rename col $\rightarrow$ $\int$ name/myname (student)
new col name old col name

① $\int$ stu_table (student)

② $\int$ s name/name (student).

Cartesian Product(X)

In DBMS, the cartesian product is an operation used in relational algebra to combine all possible pairs of rows from two tables.

It results new table that contain every possible combination of rows between two tables

Syntax: Relation1 X Relation2

It is not mandatory to have one common column in 2 tables.

$2 \times 4 = 8$ records.

Ex

student	
Sname	rollno
Sai	1
nikil	2

project	
Sname	projectid
raju	121
ramesh	122
suresh	123
Sai	124

Sname	rollno	Sname	projectid
Sai	1	raju	121
Sai	1	ramesh	122
Sai	1	suresh	123
Sai	1	Sai	124
nikil	2	raju	121
nikil	2	ramesh	122
nikil	2	suresh	123
nikil	2	Sai	124

Total
8 records.

Set Operators

In relational algebra set operators are used to manipulate and combine set of rows from tables

3 types

1. Union($\cup$)

2. Intersection($\cap$)

3. Set difference($-$)

1. Union($\cup$): Combines rows from two tables and form single row it retrieves all different rows from both tables and remove duplicate rows.

Syntax : Table1 $\cup$ Table2

EX: $A = \{1, 2, 3\} \cup B = \{3, 4, 5\}$

$A \cup B = \{1, 2, 3, 4, 5\}$

Student

name	rollno
Sai	12
Ramesh	13
Suresh	14

project

name	rollno
Sai	12
Shiva	30
Nagendra	40

o/p Student $\cup$ project

name	rollno
Sai	12
Ramesh	13
Suresh	14
Shiva	30
Nagendra	40

2.Intersection($\cap$)

Retrieves only the common rows between two tables.It returns the rows that are present in both tables.

Syntax: Table1 $\cap$ Table2

Ex1: A={1,2,3} $\cap$ B={3,4,5}

A $\cap$ B={3}

Ex2:

Name	Rollno
sai	12

3.Set Difference(-)

It retrieves rows from one tables that are not present in another table.

Syntax: Table1 - Table2

Ex1: A={1,2,3} - B={3,4,5}

A-B={1,2}

Syntax: Table1 - Table2

Ex1

Name	Rollno
ramesh	13
Suresh	14

Joins

In RDBMS, join operations are used in relational algebra to combine rows from multiple tables based on common attribute or condition.

Join operations allows you to retrieve related information from different tables by matching values in specific columns.

InnerJoin

An Inner join returns only the rows that have matching values in the specified columns of both tables

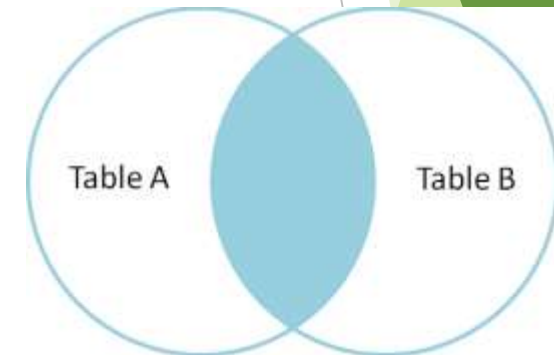
It combines the rows from that student specify join condition

Sname	Deptno
Sai	1
Shiva	3

Dname	Did
AIML	1
CSD	5

STUDENT ⋈ DEPARTMENT
(Deptno=Did)

Sname	Deptno	Dname	Did
Sai	1	AIML	1



Equi Join(Natural Join):

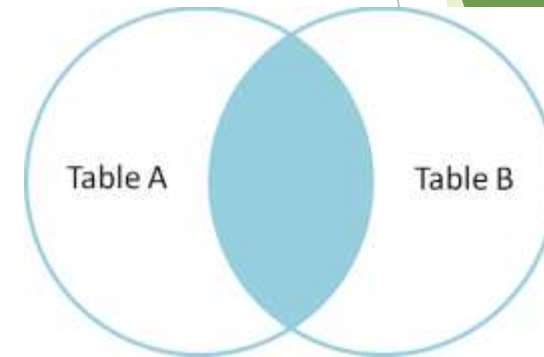
Equi join in SQL is a type of [Inner Join](#) in which two or more tables are joined by equating the common column values of the tables. The table participating in Equi join should have at least one common column on which equality operation can be performed.

Ex: Student

Rno	Sname	Cid
1	Robo	10
2	Sana	20
3	chitti	30

Course

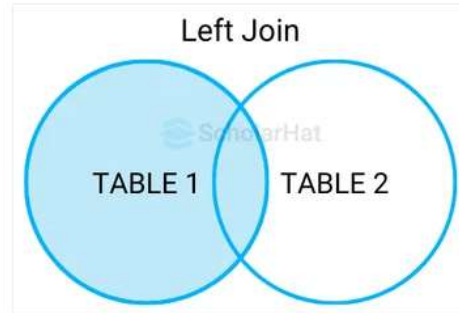
Cid	Cname
10	C
30	Java
40	C++



Sname	Cname
Robo	C
Chitti	Java

LeftJoin:

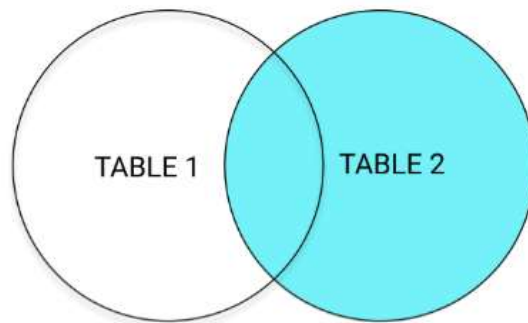
It gives record from left table even if there are no matching records in right table



sname	Cname
robo	C
sana	NULL
chitti	java

Right Join:

It gives record from right table even if there are no matching records in left table.



sname	Cname
robo	C
chitti	java
Null	C++

Full Outer Join

It gives records from both left and Right tables



sname	Cname
robo	C
sana	NULL
chitti	java
Null	C++

Theta Join

Two tables will be joined based upon the condition

Represented by $\bowtie$ (condition)

Table 1 $\bowtie$ (condition) Table 2

$\theta = (=, >, <, >=, <=)$

EX

Rno	name	marks
1	Ramesh	80
2	maheh	90
3	suresh	95

Rno	gender
3	m
2	m

marks $\bowtie$ student
 $R_1.Rno \geq R_2.Rno$

marks $\bowtie$ student
 $R_1.Rno \geq R_2.Rno$

Rno	name	marks	Rno	gender
2	maheh	90	3	m
3	suresh	95	3	m
3	suresh	95	2	m

Relational Calculus:

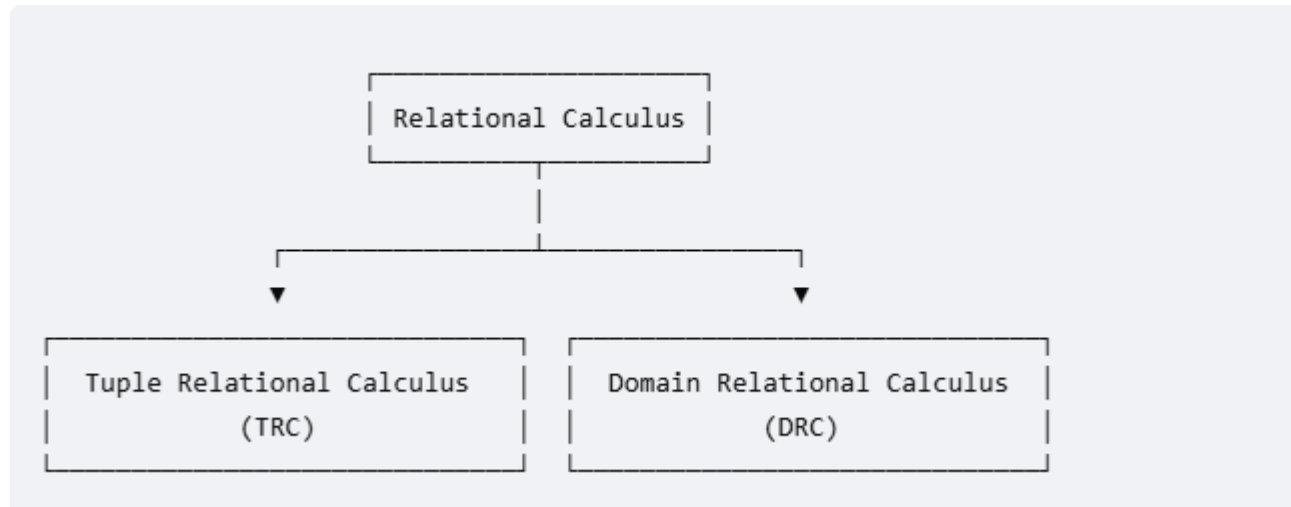
Relational Calculus is a **formal, non-procedural query language** used in relational databases to describe *what* data to retrieve rather than *how* to retrieve it.

Relational calculus tells what to do but never explains how to do

Relational calculus provides description about the query to get the result where as. Relational algebra gives the method to get the result.

TRC: proposed by codd in 1972 .works on tuples or rows

DRC: proposed by lacroix in 1977. works on domain of attributes(or columns).



1. Tuple Relational Calculus (TRC)

1. Tuple Relational Calculus (TRC)

Tuple relational calculus is a non-procedural query language.

Tuple relational calculus is used for selecting the tuples in a relation that satisfy the given condition. The result of the relation can have one or more tuples.

A query in TRC

$\{t | p(t)\}$ (or) $\{T / \text{condition}(T)\}$

t = Resulting tuple

$P(t)$ denotes predicate /condition used to fetch the tuple t .

Result of query: It is the set of all tuples t such that predicate p is true for t .

Notations used:

T is a tuple variable.

$T[A]$ denotes the value of tuple.

$t \in r$ denotes that tuple t is in relation r .

P is a formula similar to that of the predicate calculus.

Predicate calculus formula:

1. Set of attributes & constants

2. Set of comparison operators

Eg: $<, \leq, =, \neq, >, \geq$

3. Set of connectives: And ($\wedge$), or ($\vee$), not ($\neg$)
(logically)

4. Implication ($\Rightarrow$) $X \Rightarrow Y$, if X is true, then Y is true.

$$X \Rightarrow Y \equiv \neg X \vee Y.$$

5. Quantifiers: Existential Quantifiers ($\exists$) there exist and universal quantifier ($\forall$) for all.

$\exists t \in r (Q(t)) =$ "there exists" a tuple t in relation r such that predicate $Q(t)$ is true.

should satisfy some condition.

$\forall t \in r (Q(t)) \equiv Q$ is true "for all" tuples t in relation $r \rightarrow$ It should satisfy all

Ex:

FN	LN	age
Pinky	Singh	30
dolly	Singh	31
Tom	king	27
lucky	pradhas	28

Display the last name of those student where age is greater than 30.

Soln $\{ t: \text{LN} / \text{Student}(t) \text{ AND } t[\text{age}] > 30 \}$

Qp :- LN
Singh.

② Display all the details of student where last name is 'Singh'

$= \{ t \mid \text{student}(t) \text{ AND } t.LN = 'Singh' \}$

o/p:

FN	LN	age
Pinky	Singh	30
dolly	Singh	31

Domain Relational calculus(DRC)

DRC is a non procedural language.

DRC the records are filtered based on the domains(values).

DRC uses list of attributes to be selected from relation based on the condition/predicate.

Drc is same as TRC but differs by selecting the attributes rather than selecting whole tuples.

whole tuples:

* In DEC, each query is an expression of the form:

$$\{ \langle a_1, a_2, \dots, a_n \rangle \mid p(a_1, a_2, \dots, a_n) \}$$

$a_1, a_2, \dots, a_n$ represent domain variables
→ attributes.

p represents condition for fetching data.

Result of query: It is the set of all tuples $a_1, a_2, \dots, a_n$ such that predicate p is true for $a_1, a_2, \dots, a_n$ tuples.

Ex: $\{ \langle \text{article}, \text{page}, \text{Subject} \rangle \mid \in \text{Author} \wedge \text{Subject} = \text{"Database"} \}$

Notations used

* $\langle a_1, a_2, \dots, a_n \rangle \in r$, where r is relation on n attributes and $a_1, a_2, \dots, a_n$ are domain variables or domain constants.

* p is a formula similar to that of the predicate calculus -

Formula:

- Same -
1. Set of domain variables & constants
 2. Set of comparison operators $<, \leq, =, \neq, >, \geq$.
 3. Set of connectives: and ($\wedge$), or ($\vee$), not ($\neg$)
 4. Implication ($\Rightarrow$): $x \Rightarrow y$, if x is $\top$, y is true
$$x \Rightarrow y \equiv \neg x \vee y$$

3. Quantifiers: existential quantifier ($\exists$) and universal quantifier ($\forall$).

$\exists x (p(x))$ and $\forall x (p(x))$.

x is free domain variable.

Ex 2)

FN	LN	Age
Pinky	Singh	30
dolly	Singh	31
Tony	King	27
Lucky	preethy	28

Find the FN & age of std where std age is greater than 27.

$\Rightarrow \{ \langle \text{FN}, \text{age} \rangle \mid \in \text{std} \wedge \text{age} > 27 \}$

Ans:

FN	age
Lucky	28

Thank you